

Caffeine Tracker — Mini-Project Report

Project title	Caffeine Tracker — a local desktop app for logging caffeinated drinks and visualising intake against the 400 mg/day limit
Student	Mithunvel K.L
Registration number	23BMH1029
Slot	G2+TG2
Source code	https://github.com/KL-Mithunvel/CAFFINE-TRACKER
Licence	MIT

1. Title

Caffeine Tracker — a single-user Windows 11 desktop application that records every caffeinated drink (in ml), converts it to a caffeine dose (mg), and presents a dashboard of graphs and statistics, including the maximum-intake day and how often the recommended daily limit of 400 mg is exceeded.

2. Need

Caffeine is the most widely consumed stimulant, and health guidance (e.g. the FDA) treats about **400 mg per day** as a safe ceiling for healthy adults. In practice people have no idea how close they are to it:

- Caffeine content is rarely printed on a label, and varies hugely between drinks (about 32 mg per 100 ml for Red Bull versus about 10 mg per 100 ml for cola).
- Intake is spread over several drinks in different sizes, so mental arithmetic fails.
- Existing apps are cloud-based and need accounts, or are not usable without internet.
- Students with irregular study hours rely heavily on coffee and energy drinks, but have no record of the pattern (late-night intake, "above-limit" days, streaks).

The project fills this gap with a **private, offline-first, zero-account** tracker that does the unit conversion automatically and turns raw entries into meaningful statistics.

Objectives

1. Make logging a drink take under ~5 seconds using one-tap presets.
2. Compute the dose automatically: $\text{caffeine_mg} = \text{volume_ml} \times \text{caffeine_per_100ml} \div 100$.
3. Support *any* drink by looking its caffeine content up online, but never block or lose an entry when the internet is unavailable.
4. Show the user how intake compares with the daily limit (today, trends, streaks, breakdowns).
5. Keep all data on the user's own machine (single SQLite file).

3. Outcome

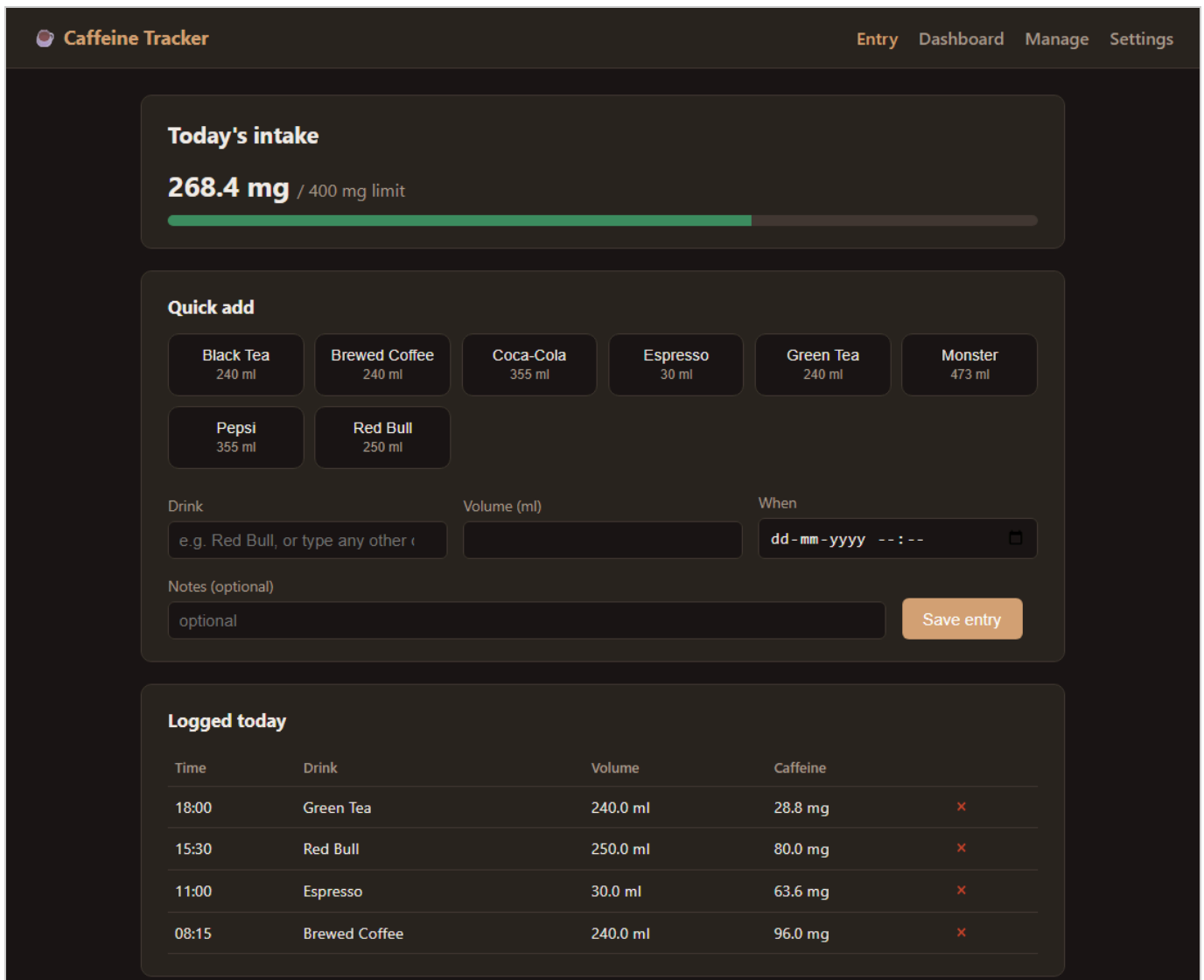
All planned features were built and tested (4 build phases plus a native-GUI migration and a PDF export). The screenshots below were taken from the running application using a synthetic dataset (`scripts/seed_fake_data.py`, 230 entries, 10 Jul – 6 Oct 2026).

3.1 Features delivered

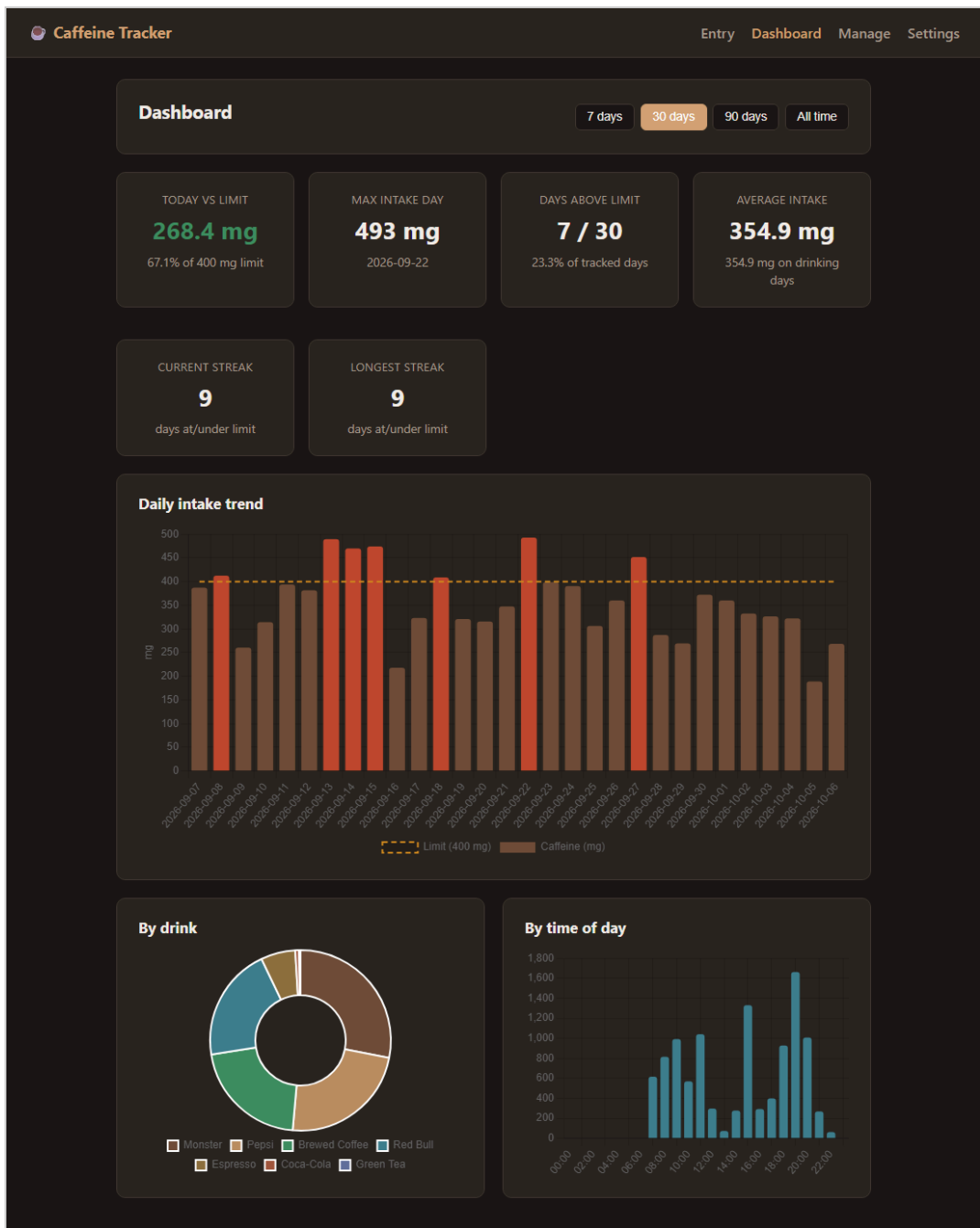
Area	What it does
Quick entry	Preset buttons (brewed coffee, espresso, black/green tea, Pepsi, Coca-Cola, Red Bull, Monster) pre-fill the typical serving size; volume, date/time and notes are editable, so late entries can be back-dated.
Unknown drinks	Typing a new drink name triggers a lookup chain: Open Food Facts API → bundled offline reference table → manual entry . The result is stored so the lookup never repeats.
Offline queue	With no internet the entry is saved at once with caffeine <i>pending</i> ; a background worker retries every few minutes and back-fills the dose when a value is found.
Dashboard	Today vs. limit, max-intake day, days above limit (count and %), average intake, current/longest streak, daily trend chart with limit line, per-drink doughnut, time-of-day histogram; 7 / 30 / 90 day / all-time ranges.
Manage	Click-to-edit or delete entries and drinks (including correcting the time of a mis-timed entry).
Settings	Editable daily limit (default 400 mg).
PDF export	One click produces a structured PDF report: summary statistics, per-drink breakdown and the full entry log (see docs/sample_report.pdf).
Native window	Flask runs locally and is shown in a native pywebview window — no browser, and a silent desktop shortcut is available.

3.2 Screenshots

Entry screen — today's progress bar, one-tap presets, entry form and today's log.



Dashboard — headline statistics and charts (days above the 400 mg limit are drawn in red).

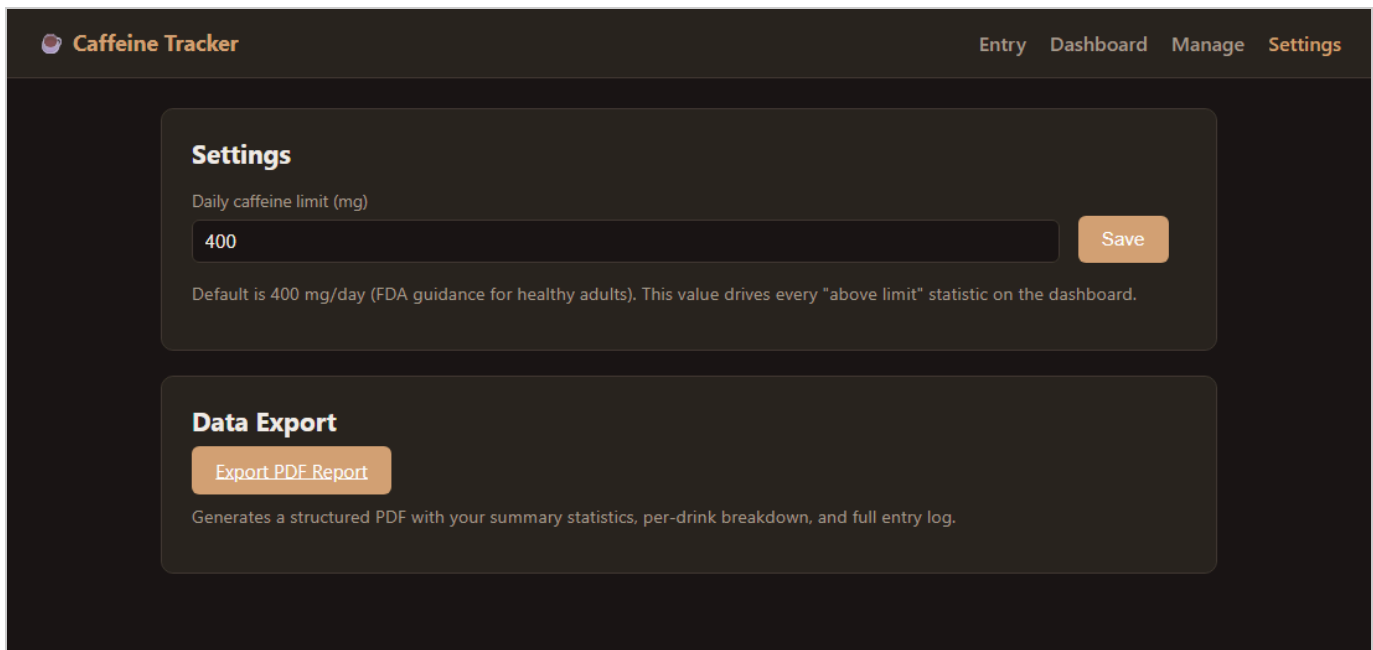


Manage entries — every entry can be edited or deleted.

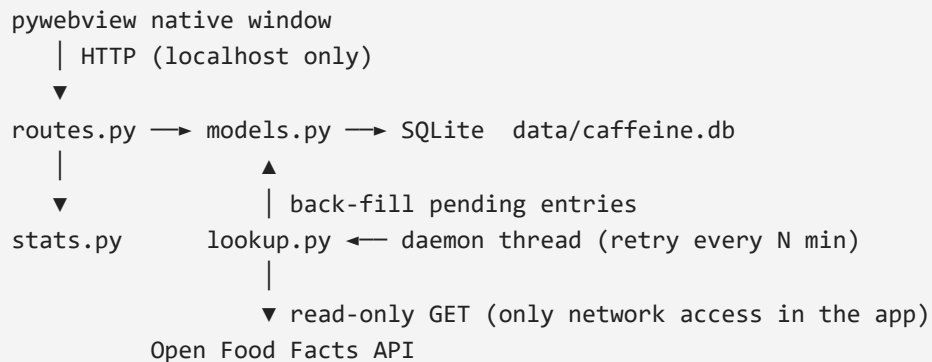
Manage entries

Date	Time	Drink	Volume (ml)	Caffeine (mg)	Notes	
2026-10-06	18:00	Green Tea	240.0	28.8		×
2026-10-06	15:30	Red Bull	250.0	80.0	pre-lab	×
2026-10-06	11:00	Espresso	30.0	63.6		×
2026-10-06	08:15	Brewed Coffee	240.0	96.0	morning cup	×
2026-10-05	20:00	Brewed Coffee	295.0	118.0		×
2026-10-05	09:00	Coca-Cola	740.0	71.04		×
2026-10-04	15:45	Pepsi	1075.0	115.03		×
2026-10-04	11:00	Red Bull	410.0	131.2		×
2026-10-04	09:30	Pepsi	710.0	75.97		×
2026-10-03	19:45	Monster	530.0	179.14		×
2026-10-03	18:30	Red Bull	460.0	147.2		×
2026-10-02	15:45	Brewed Coffee	460.0	184.0		×
2026-10-02	10:45	Monster	440.0	148.72		×
2026-10-01	20:00	Brewed Coffee	580.0	232.0		×
2026-10-01	10:30	Brewed Coffee	320.0	128.0		×
2026-09-30	18:30	Monster	445.0	150.41		×
2026-09-30	16:45	Red Bull	340.0	108.8		×
2026-09-30	07:15	Monster	335.0	113.23		×
2026-09-29	19:45	Pepsi	1130.0	120.91		×
2026-09-29	16:30	Monster	180.0	60.84		×
2026-09-29	08:45	Monster	260.0	87.88		×
2026-09-28	22:00	Brewed Coffee	160.0	64.0		×
2026-09-28	18:30	Monster	335.0	113.23		×
2026-09-28	15:00	Monster	325.0	109.85		×

Settings and PDF export



3.3 Design and architecture



Layer	Choice
Language	Python 3.12
Backend	Flask (HTTP layer kept thin)
Database	SQLite — drinks, entries, settings tables
Frontend	HTML/Jinja2, vanilla JS, Chart.js (vendored, works offline)
Desktop shell	pywebview
Reports	ReportLab (PDF)
Config / tests	YAML / pytest

Key design decisions:

- **Concentration is stored per 100 ml**, so any serving size converts with one formula.
- **caffeine_mg IS NULL means "lookup pending"** — never 0 — so statistics never silently treat unknown drinks as caffeine-free; the unresolved count is surfaced instead.

- **Entry doses are frozen at save time:** editing a drink's concentration later does not rewrite history (only pending entries are back-filled).
- **Business logic lives in the backend** (`models.py`, `stats.py`, `lookup.py` import no Flask), keeping it unit-testable and the JavaScript thin.
- **Privacy:** the server binds to `127.0.0.1`; the only outbound traffic is a read-only caffeine lookup.

Full detail: [REQUIREMENTS.md](#), [PROJECT_PLAN.md](#), [SCHEMA.md](#).

3.4 Testing and verification

Check	Result
<code>pytest tests/ (models, stats, lookup, routes)</code>	55 passed
<code>python -m py_compile app/*.py main.py</code>	OK
Live run of the Flask app on a seeded database	Entry, Dashboard, Manage, Settings pages and PDF export all returned correctly (screenshots above)
Offline flow (pending → manual resolve → back-fill)	Verified in an end-to-end smoke test

3.5 Limitations

- The live Open Food Facts lookup is covered by mocked-HTTP tests only; it was not verified against the real API in the development sandbox.
- Caffeine values for brands are typical figures and may differ between products and countries; the user can correct any value on the Manage screen.
- Single user, single machine; no sync or mobile client.
- Developed and tested on Python 3.11; the target is 3.12+.

4. Future perspectives

1. **Caffeine decay model** — estimate the caffeine still active in the body (about a 5-hour half-life) and warn about late-evening intake that may affect sleep.
2. **Smart reminders** — a notification when a new entry would take the user past the limit.
3. **Wider drink database** — barcode scanning and a larger curated reference list (brands, café menus, decaf, supplements).
4. **Optional sync and mobile companion** — an encrypted backup or phone app, while keeping local-first storage as the default.
5. **Health correlations** — compare intake with sleep or heart-rate data imported from a wearable.
6. **Personalised limits** — adjust the daily limit for body weight, pregnancy or medical advice; weekly/monthly goals for gradually cutting down.
7. **Packaging** — a one-file Windows installer (PyInstaller) and a Linux build, and a migration to `uv` for dependency management.

5. Conclusion

Caffeine Tracker meets its objectives: it removes the arithmetic from caffeine tracking, works without internet, keeps all data private on the user's machine, and turns a simple drink log into clear, actionable statistics. The modular, tested backend makes the future extensions above straightforward to add.

6. How to run

```
git clone https://github.com/KL-Mithunvel/CAFFINE-TRACKER.git
cd CAFFINE-TRACKER
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
python main.py
```